

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Typestate Checker: Análisis de una herramienta de verificación de Object Protocols

Tesis de Licenciatura en Ciencias de la Computación

Manuel Ignacio Lakowsky

Director: Sebastián Uchitel

Buenos Aires, 2026

TYPESTATE CHECKER: ANÁLISIS DE UNA UNA HERRAMIENTA DE VERIFICACIÓN DE OBJECT PROTOCOLS

Diversas herramientas y técnicas se han desarrollado a lo largo del tiempo para detectar potenciales errores en programas. En particular, los sistemas de tipado ...

Palabras claves: Guerra, Rebelión, Wookie, Jedi, Fuerza, Imperio (no menos de 5).

TYPESTATE CHECKER: AN ANALYSIS OF AN OBJECT PROTOCOLS CHECKER TOOL

In a galaxy far, far away, a psychopathic emperor and his most trusted servant – a former Jedi Knight known as Darth Vader – are ruling a universe with fear. They have built a horrifying weapon known as the Death Star, a giant battle station capable of annihilating a world in less than a second. When the Death Star's master plans are captured by the fledgling Rebel Alliance, Vader starts a pursuit of the ship carrying them. A young dissident Senator, Leia Organa, is aboard the ship & puts the plans into a maintenance robot named R2-D2. Although she is captured, the Death Star plans cannot be found, as R2 & his companion, a tall robot named C-3PO, have escaped to the desert world of Tatooine below. Through a series of mishaps, the robots end up in the hands of a farm boy named Luke Skywalker, who lives with his Uncle Owen & Aunt Beru. Owen & Beru are viciously murdered by the Empire's stormtroopers who are trying to recover the plans, and Luke & the robots meet with former Jedi Knight Obi-Wan Kenobi to try to return the plans to Leia Organa's home, Alderaan. After contracting a pilot named Han Solo & his Wookiee companion Chewbacca, they escape an Imperial blockade. But when they reach Alderaan's coordinates, they find it destroyed - by the Death Star. They soon find themselves caught in a tractor beam & pulled into the Death Star. Although they rescue Leia Organa from the Death Star after a series of narrow escapes, Kenobi becomes one with the Force after being killed by his former pupil - Darth Vader. They reach the Alliance's base on Yavin's fourth moon, but the Imperials are in hot pursuit with the Death Star, and plan to annihilate the Rebel base. The Rebels must quickly find a way to eliminate the Death Star before it destroys them as it did Alderaan (aprox. 200 palabras).

Keywords: War, Rebellion, Wookie, Jedi, The Force, Empire (no menos de 5).

AGRADECIMIENTOS

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce sapien ipsum, aliquet eget convallis at, adipiscing non odio. Donec porttitor tincidunt cursus. In tellus dui, varius sed scelerisque faucibus, sagittis non magna. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Mauris et luctus justo. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Mauris sit amet purus massa, sed sodales justo. Mauris id mi sed orci porttitor dictum. Donec vitae mi non leo consectetur tempus vel et sapien. Curabitur enim quam, sollicitudin id iaculis id, congue euismod diam. Sed in eros nec urna lacinia porttitor ut vitae nulla. Ut mattis, erat et laoreet feugiat, lacus urna hendrerit nisi, at tincidunt dui justo at felis. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Ut iaculis euismod magna et consequat. Mauris eu augue in ipsum elementum dictum. Sed accumsan, velit vel vehicula dignissim, nibh tellus consequat metus, vel fringilla neque dolor in dolor. Aliquam ac justo ut lectus iaculis pharetra vitae sed turpis. Aliquam pulvinar lorem vel ipsum auctor et hendrerit nisl molestie. Donec id felis nec ante placerat vehicula. Sed lacus risus, aliquet vel facilisis eu, placerat vitae augue.

A mi persona favorita.

Índice general

1.. Introducción 1
1.1. Estructura de la Tesis 1

2.. Trabajos Relacionados 3
2.1. Typestates y Object Protocols 3
2.1.1. Inicios 3
2.1.2. Programación Orientada a Objetos 3
2.1.3. Uso de Protocolos 4
2.2. Plaid 4
2.3. Mungo 5
2.3.1. Sintaxis y funcionalidad 6
2.3.2. Implementación 6
2.3.3. Estado del proyecto 7

3.. Java Typestate Checker 8
3.1. Definición de Protocolos 8
3.1.1. Estados Descartables 9
3.1.2. Múltiples Destinos 9
3.2. Aliasing 10
3.3. Anotaciones Adicionales 11
3.3.1. Nullable 11
3.3.2. Requires 11
3.3.3. Ensures 12
3.4. Subtipado 13
3.5. Impacto 15
3.6. Limitaciones 15
3.7. Implementación 15
3.7.1. Diferencias con la versión actual 16
3.7.2. Visión general 16
3.7.3. Primera Fase: Inferencia de Tipos 16
3.7.4. Segunda Fase: Verificación de Errores 23
3.7.5. Checker Framework 24

4.. Experimentación 27
4.1. Protocolos Compuestos 27
4.2. Correctitud vs. Orden de Métodos 32
4.3. Intervención Humana 34
4.4. Subtipado 38
4.5. Estado Vs Typestate 38
4.6. Casos Bordes 38
4.7. Aliasing 38
4.8. Excepciones 38

1. INTRODUCCIÓN

A lo largo de los años, diversas herramientas se han desarrollado para detectar lo antes posible errores en programas. Desde sistemas de tipado y análisis estático de código, hasta frameworks de testing unitario y/o automático, se busca agregar robustez al software y permitirle al desarrollador tener más confianza sobre lo que produce.

Estos potenciales errores en programas pueden ser agrupados en diferentes categorías. Por ejemplo, los errores aritméticos, que ocurren al realizar operaciones matemáticas indebidas, tales como dividir por cero, o errores de punteros, que suceden al desreferenciar punteros nulos. Un grupo particular de errores son los relacionados a los ***Object Protocols*** en lenguajes de programación orientados a objetos. En pocas palabras, los protocolos de los objetos son los que determinan las secuencias de métodos válidas para los mismos. Uno de los ejemplos típicos de un protocolo se encuentra en los lectores de archivos, donde antes de leer hay que abrir el descriptor del archivo, y al finalizar hay que cerrarlo. Otro ejemplo es el de los `Iterator`s en Java[21], donde el método `remove()` solamente puede ser llamado una única vez luego de cada llamado a `next()`. Pensar en *object protocols* no es inusual, y múltiples instancias de uso y definición de estos protocolos pueden encontrarse en varios proyectos open source[4].

De no cumplir con el protocolo de un objeto al usar sus métodos pueden generarse excepciones inesperadas, y en algunos casos el comportamiento de los mismos puede ser indefinido. Es por esto que cobran importancia las herramientas de detección de incumplimiento de *object protocols*, las cuales, analizando el código estática o dinámicamente, permiten avisarle lo antes posible al desarrollador de un potencial error.

En este trabajo enfocaremos nuestro estudio en una de estas herramientas llamada **Java Typestate Checker**[19]. *JaTyC* analiza código de Java estáticamente para determinar si el uso de un objeto por parte de algún cliente rompe el protocolo del mismo. Estos protocolos son definidos por los mismos programadores a través de **Typestates**, los diferentes estados por los que puede pasar el objeto y los métodos habilitados en cada uno.

Está basada en **Mungo**[25], un conjunto de herramientas de investigación universitarias centradas en la verificación de protocolos, agregando mejoras y funcionalidades innovadoras[18]. A su vez, está escrita como plugin y mencionada en la documentación oficial del **Checker Framework**[8], en el que confían empresas tales como Amazon[2][3] y que ha sido utilizado en incontables publicaciones. En ese sentido, *JaTyC* se posiciona como uno de los analizadores más flexibles y contemporáneos para *object protocols* en Java.

A lo largo de esta investigación realizaremos múltiples pruebas con la herramienta, aprendiendo sobre su poder de expresión, las garantías que provee y sus limitaciones. Buscamos mejorar el entendimiento del alcance de *JaTyC*, y a su vez de la verificación de *object protocols* a través de typestates en general.

1.1. Estructura de la Tesis

En el capítulo 2 se dará un poco de contexto teórico y práctico alrededor de los *object protocols* y typestates. Se mencionarán publicaciones relacionadas y herramientas/lenguajes predecesoras a *JaTyC*.

El capítulo 3 se centrará en la herramienta, explicando qué es y cómo se usa. Se mencionará además cómo funciona internamente, cuál es su algoritmo de análisis estático de código, y cómo se apoya en el *Checker Framework*.

El capítulo 4 recopilará los hallazgos de la experimentación sobre *JaTyC*. Se dividirá en distintas secciones según los aspectos o funcionalidades de la herramienta a poner bajo la lupa.

En el capítulo 5 se hará un resumen de lo aprendido, se establecerán conclusiones, y se discutirá trabajo futuro.

2. TRABAJOS RELACIONADOS

2.1. Typestates y Object Protocols

2.1.1. Inicios

La noción de **typestate** fue introducida en 1986 por Strom y Yemini [23]. A pesar de que ya en esa época existían mecanismos de prevención de errores en programas, tales como los sistemas de tipado o reglas basadas en scopes estáticos, los autores identificaron una limitación en los mismos. Específicamente, no estaban diseñados para detectar ciertas operaciones prohibidas en algunos contextos, tales como desreferenciar un puntero apuntando a memoria ya liberada, o utilizar una variable antes de ser inicializada. Para cubrir esa necesidad definieron los **typestates**, los cuales intentan modelar el estado en el que se encuentra un objeto a la hora de realizar una operación. Mientras que los tipos restringen las operaciones permitidas para un dato/objeto, y los scopes el acceso a variables fuera de su ciclo de vida, los typestates intentan restringir el orden en el cual las operaciones suceden bajo un contexto particular.

El funcionamiento de los typestates es el siguiente. Dado un tipo, se le asocia un conjunto de typestates, y se define para cada uno un conjunto de operaciones permitidas. A su vez, se definen reglas de transición entre ellos a través de estas operaciones. En cualquier punto del programa, un objeto estará en alguno de estos typestates y podrán ser aplicadas solamente sus operaciones permitidas. Al interactuar con él, podrá transicionar a otro typestate según las reglas definidas, cambiando el conjunto de operaciones disponibles.

El paper menciona además conceptos que herramientas contemporáneas como *JaTyC* implementan, tales como el de **completitud de protocolo**, donde se espera que los objetos lleguen a un estado final antes de que el programa termine. Según los autores, una ejecución es *typestate-correcta* si y solo si todas las operaciones realizadas en objetos son permitidas por el typestate que las acompaña, y todos los objetos completan su protocolo.

Finalmente, el trabajo establece las definiciones formales alrededor de los typestates para trabajar con ellos. Con las mismas, presenta un algoritmo para analizar estáticamente el cumplimiento de protocolos en programas.

2.1.2. Programación Orientada a Objetos

Estas ideas fueron posteriormente extendidas por Bierhoff y Aldrich [6], llevándolas hacia el mundo de la programación orientada a objetos.

Por un lado, introducen y formalizan la noción de *refinamiento de estados*, utilizada cuando se quiere hacer herencia de un tipo ya protocolado. Si el subtipo quiere agregar nuevos typestates a su protocolo, debe asegurarse necesariamente que se sigan cumpliendo las restricciones del protocolo del supertipo que extiende. En otras palabras, toda secuencia de llamadas de métodos válidas en el tipo padre debe mantenerse en el protocolo del tipo hijo. El refinamiento de estados habla sobre cómo un subconjunto de estados en el subtipo son "subcasos" de un solo estado en el supertipo, los cuales pueden agregar nuevas secuencias de métodos válidas al protocolo, pero no quitar las preexistentes. En cierto sentido, si modelamos el protocolo de un objeto como una máquina de estados finita, el autómata del subtipo debe preservar el comportamiento del autómata del supertipo.

Por otro lado, introducen la idea de *dimensiones de estados*. Esta noción busca dar respuesta a situaciones en las que conceptualmente se considera que un objeto está en varios typestates a la vez, atados a variables completamente independientes. Estos subconjuntos de estados son referidos en el trabajo como *ortogonales*. Por ejemplo, observemos la interfaz de la figura 2.1:

```
1 public interface Iterator {
2     public boolean hasNext();
3     public int next();
4     public boolean canPrint();
5     public void print();
6 }
```

Fig. 2.1: Ejemplo de tipo con typestates ortogonales

Puede observarse que el comportamiento de la interfaz depende de si se puede seguir iterando sobre la colección y de si actualmente se puede imprimir. Asumiendo que ambas condiciones son independientes entre sí, definir un protocolo que correctamente especifique las cadenas de métodos válidas puede resultar tedioso y conceptualmente incorrecto. Esto es porque, para modelar el protocolo lo más detallado posible, se necesitaría un typestate por cada elemento del producto cartesiano entre los conjuntos $\{PuedoImprimir, NoPuedoImprimir\}$ y $\{TengoSiguiente, NoTengoSiguiente\}$. Puede verse fácilmente que, con un tipo que tenga un gran número de variables ortogonales de las cuales dependen los typestates, la definición del protocolo puede dispararse en tamaño. A su vez, puede que tener un estado como *TengoSiguienteImprimible* no tenga mucho sentido.

Para solventar esto, el paper define que el objeto está en varias *dimensiones de estados* ortogonales a la vez, las cuales permiten distintas secuencias de llamadas de métodos. Para nuestro ejemplo, la interfaz puede encontrarse en el estado (*PuedoImprimir*, *TengoSiguiente*), permitiéndole acceso a los métodos `print()` y `next()`. Esto evita la explosión de cantidad de estados, mejora la legibilidad y claridad del protocolo, y también permite acercarnos a utilizar typestates para herencia múltiple (definiendo los protocolos heredados como ortogonales).

2.1.3. Uso de Protocolos

Finalmente, en el trabajo de Beckman et al. [5] se realizó un estudio sobre el uso de *object protocols* en proyectos open source y la librería estándar de Java. En él se intenta describir qué tanto se suelen definir protocolos, cuánto se utilizan y se respetan por código cliente, y qué estructura suelen tener. Se descubrió que, a lo largo del código analizado, aproximadamente 7 % de todos los tipos definían protocolos, y más del 10 % de las clases eran clientes de tipos protocolados. Dado que la muestra de código analizada fue significativa, esto nos indica que las herramientas de verificación de protocolos tales como *JaTyC* son potencialmente aplicables a numerosos proyectos.

2.2. Plaid

A raíz del surgimiento del concepto de typestates, se ideó un nuevo paradigma de programación: la Programación Orientada a Typestates (TSOP)[11]. Bajo este paradigma,

el código no solo es modelado alrededor de clases sino también sus typestates y protocolos. En otras palabras, pasan a ser *first-class citizens* (entidades de primer orden).

Plaid[24][1] es uno de los primeros lenguajes de programación orientada a typestates, desarrollado por la Universidad Carnegie Mellon. En Plaid, los objetos siempre se encuentran en un typestate particular, los cuales se definen en forma similar a las clases tradicionales (con métodos, variables, subestados, etc). Estos pueden transicionar durante la ejecución del programa entre un typestate y otro, modificando su interfaz y comportamiento.

Un ejemplo puede observarse en la Figura 2.2. En ella, se define la interfaz `File` a través de los dos typestates `ClosedFile` y `OpenFile`. Mientras que la variable `filename` está disponible en todos los estados, hay métodos exclusivos para cuando los objetos de tipo `File` estén abiertos o cerrados. A su vez, se definen en los métodos las transiciones entre typestates, como por ejemplo en `[ClosedFile≫OpenFile]`, transicionando un objeto en el estado cerrado al abierto. Esto puede hacerse tanto para el objeto receptor del método como para sus argumentos.

```
1  state File {
2      val String filename;
3  }
4
5  state ClosedFile = File with {
6      method void open() [ClosedFile≫OpenFile];
7  }
8  state OpenFile = File with {
9      private val CFile fileResource;
10
11     method int read();
12     method void close() [OpenFile≫ClosedFile];
13 }
```

Fig. 2.2: Ejemplo de definición de clase y typestates con Plaid

Entre otras funcionalidades, Plaid cuenta con un sistema de permisos de acceso[7] para intentar solucionar el problema de la verificación de protocolos en la presencia de *aliasing*. Finalmente, similar a *JaTyC*, Plaid cuenta con un analizador estático que permite verificar el uso correcto de los typestates y los protocolos de los objetos en tiempo de compilación.

2.3. Mungo

Mungo[25][16] es una herramienta predecesora de *JaTyC* para Java que, similarmente, verifica estáticamente que se sigan los protocolos de objetos. Esto lo consigue a través de la abstracción de typestates, modelando los diferentes estados por los que pasa una clase, y reporta errores cuando código cliente no respeta el protocolo de las mismas. En la documentación oficial de *JaTyC*[20][17], los autores hacen hincapié en que Mungo es la herramienta principal en la que se basaron para su desarrollo, incluso mencionando que el plan inicial era simplemente extenderla con nuevas funcionalidades. En cambio, *JaTyC* terminó siendo una herramienta independiente, y es de interés notar las diferencias entre ambas[18].

2.3.1. Sintaxis y funcionalidad

Comenzando por la funcionalidad y sintaxis, las dos herramientas son bastante similares. Las definiciones de los protocolos y sus typestates son escritas en archivos separados con extensión *.protocol* (ver Figura 2.3). Luego, para referenciar un protocolo desde una clase se la anota con "`@Typestate(<protocolo>)`". Cuando el código fuente de Java es compilado, primero es analizado estáticamente por el mismo sistema de tipado del lenguaje, y luego por el de Mungo. Si una clase está asociada a un protocolo, sus clientes deben respetar el correcto orden de llamados de sus métodos. De lo contrario, la herramienta reporta un error.

```
typestate StackProtocol {
    Empty = {
        void push(int): NonEmpty,
        void deallocate(): end
    }
    NonEmpty = {
        void push(int): NonEmpty,
        int pop(): Unknown
    }
    Unknown = {
        void push(int): NonEmpty,
        Check isEmpty(): <EMPTY: Empty, NONEMPTY: NonEmpty>
    }
}
```

Fig. 2.3: Ejemplo de definición de protocolo para un Stack en Mungo

Los protocolos se escriben con una sintaxis muy similar a la de *JaTyC*, e incluso idéntica en muchos casos. En la Figura 2.3 puede observarse un ejemplo oficial de cómo se vería un protocolo para una clase `Stack`, que permite apilar infinitamente y solo permite desapilar si no está vacío. Cuenta con tres typestates, uno para cuando la pila está vacía `Empty`, uno para cuando la pila tiene algún elemento `NonEmpty`, y otro para cuando no se sabe el estado actual de la misma `Unknown`. Cada typestate describe la signatura de los métodos permitidos en cada uno, y el (o los) typestate al que pasaría el objeto de llamarlos. Al ser instanciada, la pila comienza siempre en el primer typestate definido en el protocolo, en este caso `Empty`, y finaliza su protocolo al transicionar al typestate reservado `end` llamando al método `deallocate()`.

Un aspecto en el que *JaTyC* mejora con respecto a su antecesor es la funcionalidad. Información detallada puede encontrarse en el cuadro comparativo oficial de la herramienta[18], pero las principales utilidades que agrega son los estados *droppable*, el subtipado, y verificación de typestates en argumentos y valores de retorno de métodos. A su vez, los autores recalcan que Mungo falla al manejar ciertos casos borde, relacionados con valores nulos, linealidad de las variables y corroborar la finalización de los protocolos. Profundizaremos más en todos estos conceptos en el próximo capítulo.

2.3.2. Implementación

Con respecto a la implementación de ambas herramientas, Mungo está creada sobre el framework **JastAdd**[13]. Este es un sistema de meta-compilación fácilmente extensible, que le permite a sus usuarios crear herramientas de análisis estático customizables. A pesar

de haber sido utilizado para múltiples proyectos[14], JastAdd no se mantiene activamente, con solamente 3 releases desde 2020.

En comparación, *JaTyC* está implementada en el **Checker Framework**[8], que extiende el sistema de tipado de Java. El framework le permite a los desarrolladores incluir numerosos plug-ins, o checkers, a sus proyectos, los cuales realizan diferentes tipos de verificaciones en tiempo de compilación. Uno de los checkers más conocidos del framework es el *Nullness Checker*, el cual verifica que no pueda suceder ninguna excepción de `null pointers` durante ejecuciones. Cuenta con varios otros plug-ins útiles, tales como el *SQL Quotes Checker* o el *Regex Checker*. Es un framework mantenido activamente, y es utilizado en la industria, por ejemplo por Amazon con sus checkers propios *KMS Compliance Checker*[3] y *Crypto Policy Compliance Checker*[2].

Por otro lado, Mungo está desarrollada en Java. Parte de la razón por la que los autores de *JaTyC* decidieron hacerla una herramienta independiente, y no una extensión de Mungo, fue poder escribirla en Kotlin[15]. Ellos destacan que la interoperabilidad entre ambos lenguajes (que permite reutilizar librerías de Java), junto con la sintaxis más concisa de Kotlin y sus funcionalidades de seguridad, hicieron más productivo el desarrollo.

2.3.3. Estado del proyecto

Finalmente, el repositorio de Mungo[26] muestra que el último commit se realizó en 2020, por lo que podemos asumir que el desarrollo de la herramienta está inactivo. Por otro lado, *JaTyC* estuvo siendo actualizada hasta recientemente, con su último commit subido en 2025.

3. JAVA TYPESTATE CHECKER

Java Typestate Checker [19] es una herramienta de verificación de código Java para protocolos de objetos. Mediante el análisis estático del programa, detecta errores relacionados con el orden de las llamadas a métodos, el consumo debido de los recursos y la herencia de protocolos, entre otros aspectos. La herramienta está escrita en Kotlin [15] como plug-in del Checker Framework [8], y se destaca por ser una de las herramientas de análisis de *object protocols* más flexibles y actualizadas para Java. Por estas razones la seleccionamos para analizar el poder de expresión y las limitaciones de la verificación de protocolos. A lo largo de este capítulo repasaremos las principales características de *JaTyC* con ejemplos, y detallaremos su arquitectura e implementación.

3.1. Definición de Protocolos

JaTyC permite a sus usuarios definir protocolos para sus clases a través de **typestates**, los diferentes estados en los cuales una instancia puede encontrarse durante su ciclo de vida. Dentro de cada typestate se establece la interfaz de las instancias que se encuentran en él, definiendo los métodos disponibles y las transiciones resultantes de llamarlos.

```
typestate Iterator {
  HasNext = {
    boolean hasNext(): <true: Next, false: end>
  }
  Next = {
    boolean hasNext(): <true: Next, false: end>,
    Object next(): HasNext
  }
}
```

Fig. 3.1: Definición de protocolo simple para un Iterador

```
import jatyc.lib.Typestate;

@Typestate("Iterator")
public interface Iterator {
  public boolean hasNext();
  public int next();
}
```

Fig. 3.2: Suscripción de protocolo a través de la anotación `@Typestate`

Un ejemplo simple de un protocolo puede observarse en la Figura 3.1. En él se definen dos typestates para una clase `Iterator`: `HasNext` y `Next`. Al ser el primer estado definido, `HasNext` es el inicial para cualquier nueva instancia de la clase, y cuenta solamente con el método `hasNext()` disponible. Al ser llamado, el objeto puede transicionar a diferentes estados según el resultado booleano retornado: al estado `Next` de ser verdadero, o al estado reservado `end` de ser falso, que señala la terminación del protocolo. Una vez finalizado el

protocolo, no se podrá llamar a ninguno de sus métodos (con la excepción de los métodos *anytime*, que trataremos en la sección 3.2).

De transicionar a `Next`, el iterador podrá llamar a ambos métodos, `hasNext()` y `next()`, donde el último devuelve el siguiente elemento de la colección y lleva a la instancia de vuelta al estado inicial. En resumen, el protocolo indica que el cliente solamente puede pedir el siguiente elemento si tiene la certeza de que existe, y que de no haber más elementos el objeto termina su ciclo de vida. Los clientes del protocolo deberán respetar este comportamiento; de lo contrario, *JaTyC* alertará un error.

Anotando una clase con `@Typestate(<protocolo>)` se la suscribe al protocolo, como muestra la Figura 3.2.

3.1.1. Estados Descartables

Los **Droppable States**, o "estados descartables", son una funcionalidad nueva introducida por la herramienta en comparación con Mungo. Estos ofrecen al programador una alternativa a la hora de indicar la finalización del protocolo de un objeto.

```
typestate Animal {
  Init = {
    void move(): Moved,
    drop: end
  }
  Moved = {
    void eat(): Init
  }
}
```

Fig. 3.3: Protocolo con estados descartables

En la Figura 3.3 puede observarse cómo funcionan. El estado inicial `Init` tiene declarado un único método llamable, `move()`, y además es descartable, lo cual se indica con la notación `drop: end`. La misma indica que, cuando finalice el ciclo de vida del objeto, si este se encuentra en el estado `Init` transicionará automáticamente a `end`, finalizando su protocolo.

Los estados descartables permiten mayor flexibilidad al definir protocolos, sobre todo cuando no existe un método o condición clara que lleve al objeto a `end`. Cualquier estado puede ser descartable, y varios estados pueden serlo a la vez.

3.1.2. Múltiples Destinos

Al definir un protocolo, hay situaciones en las que podría ser deseable expresar, para un mismo método, transiciones a typestates diferentes según el resultado de la llamada.

JaTyC permite definir múltiples destinos para resultados booleanos o enumeraciones. Por ejemplo, en la Figura 3.1 podemos observar cómo para el método `hasNext()` se define que el objeto pasará al estado `Next` o a `end` según si el método devuelve verdadero o falso. Por otro lado, en la Figura 3.4 se aprecia un protocolo para un lector de archivos. En él, el estado inicial `Init` define un método `open()` que devuelve una enumeración de tipo `FileStatus`, con los valores posibles `OK` y `ERROR`. Según el resultado devuelto, el objeto podrá leer el archivo abierto o finalizará su protocolo.

```
typestate FileProtocol {
  Init = {
    FileStatus open(): <OK: Read, ERROR: end>
  }
  Read = {
    String read(): Close
  }
  Close = {
    void close(): end
  }
}
```

Fig. 3.4: Protocolo con múltiples destinos a través de enumeraciones

3.2. Aliasing

Una de las situaciones problemáticas al analizar código estáticamente es la existencia de varias referencias vivas a un mismo objeto, conocido como **aliasing**. En particular, dificulta (y a veces imposibilita) asegurarse de que una referencia en otra parte del código no haya modificado el estado del objeto, invalidando lo que el analizador asume como correcto y potencialmente arruinando su resultado.

Para solventarlo, *JaTyC* impone una restricción dura sobre el aliasing, obligando a los programadores a usar objetos protocolados **linealmente**. Esto significa que, para un objeto protocolado, una única referencia podrá modificar su typestate, llamada *referencia lineal*. Esto limita el poder de expresión de los desarrolladores, pero permite a la herramienta verificar que los objetos sigan y completen sus protocolos. Las demás referencias, denominadas *referencias compartidas*, tendrán acceso solamente a los métodos *anytime*.

Los **Anytime Methods**, o "métodos de cualquier momento", son todos los métodos definidos por una clase que no están mencionados en su protocolo, o todos los métodos de una clase en caso de no tener protocolo. Estos pueden ser llamados sobre un objeto en cualquier punto de la ejecución del programa, sin importar el estado en el que se encuentre. Dado que estos métodos se interpretan como ajenos al protocolo, y para preservar la linealidad, no pueden modificar los valores ni el typestate de los colaboradores internos de la instancia.

```
public class Main {
  public static void main(String args[]) throws Exception {
    Stack s1 = new Stack(5);
    Stack s2 = s1;
    s2.push(5);
    s1.push(5);
  }
}
```

```
Main.java:6: error: Cannot call [push] on Shared{Stack}
    s1.push(5);
```

Fig. 3.5: JaTyC reportando un error al llamar métodos protocolados en una referencia compartida

Una variable pierde la linealidad de su referencia, de tenerla, cuando esta se asigna a

otra variable, parámetro o colaborador interno. Es decir, asignar una referencia lineal a una variable nueva "transmite la linealidad" de la vieja a la nueva, mientras que la vieja pasa a ser compartida.

En la Figura 3.5 puede observarse un ejemplo de código cliente que utiliza un objeto protocolado de tipo `Stack`. Este tiene un protocolo muy simple, similar al de la Figura 2.3, el cual incluye al método `push()` (en otras palabras, no es un método *anytime*). El código cliente comienza definiendo una variable `s1` y asignándole una nueva instancia de `Stack`, lo que la convierte en una referencia lineal. Acto seguido, se define una nueva variable `s2` y se la hace apuntar al mismo objeto. Esto transfiere la linealidad de `s1` a `s2`, y `s1` pasa a ser una referencia compartida, sin acceso a los métodos protocolados. Notemos que, al correr la herramienta, esta reporta como error la llamada al método `push()` desde la referencia compartida `s1`, y no desde la lineal `s2`.

3.3. Anotaciones Adicionales

Además de la definición de protocolos, tanto en los archivos *.protocol* como con la anotación `@Typestate` para suscribirse a ellos, *JaTyC* provee otras anotaciones útiles para definir el comportamiento esperado de los programas.

3.3.1. Nullable

Por defecto, *JaTyC* considera a todas las variables de un programa como no anulables, a diferencia del sistema de tipos de Java. Para indicar que una variable puede tomar el valor `null`, se utiliza la anotación `@Nullable`, como se muestra en la Figura 3.6. De no utilizarla y asignar `null` a una variable no anulable, la herramienta reportará un error. A través de este mecanismo, el checker previene errores típicos de Java, como los de tipo *Null Pointer*.

Esta anotación puede utilizarse en la definición de variables y colaboradores internos, en argumentos de funciones y en valores de retorno.

```
1 public @Nullable int next() {
2   return this.items[this.index++];
3 }
```

Fig. 3.6: Uso de la anotación `@Nullable` para el método `next()` de un iterador

3.3.2. Requires

La anotación `@Requires` puede utilizarse en los parámetros de un método para indicar en qué typestate (o conjunto de typestates) se espera que se encuentre el objeto provisto. Un ejemplo de esto puede observarse en la Figura 3.7, donde se muestran un par de métodos para un controlador de un robot. En ellos se requiere que el robot pasado por parámetro esté obligatoriamente en determinados estados.

De no proveer esta anotación, *JaTyC* asume que el objeto recibido por el método es una referencia compartida, y permite utilizar solamente sus métodos *anytime*. En cambio, de proveerla, se debe tener en cuenta la finalización del protocolo del objeto. Es decir, cuando se salga del *scope* del método se perderá la única referencia lineal al objeto, y por

ende su protocolo debe completarse antes (a no ser de utilizar la anotación `@Ensures`, que trataremos en la próxima sección).

```
public void moveRight(@Requires("TopLeft") Robot robot) {
  robot.moveRight();
}

private void moveLeft(@Requires({"TopRight", "BotRight"}) Robot robot) {
  robot.moveLeft();
}
```

Fig. 3.7: Uso de la anotación `@Requires` para un solo typestate y para un conjunto de typestates

3.3.3. Ensures

Por último, la herramienta provee la anotación `@Ensures`, utilizada para indicar el typestate (o conjunto de typestates) en el que se encuentra el valor de retorno de un método. Un ejemplo puede observarse en la Figura 3.8, donde se ve un método *getter* de un controlador de robots. En él se define que el valor devuelto será una referencia lineal a un robot que se encuentra exactamente en el estado `TopLeft`. Al retornar, el código cliente podrá acceder a los métodos protocolados del robot.

De no utilizar esta anotación, se asume que la referencia devuelta será compartida, y solamente sus métodos *anytime* estarán disponibles. Además, de no utilizarla y devolver un objeto que tiene protocolo (como el `Robot` de la Figura 3.8), se requiere que el método lo finalice antes de retornar (ya sea a través de una transición a `end` o de un estado descartable). De no ser así, se perdería la única referencia lineal al salir del *scope*, y el objeto nunca llegaría a completar su protocolo. Por ejemplo, para retornar un lector de archivos, se espera que antes de finalizar el método se haya cerrado el archivo.

```
public @Ensures("TopLeft") Robot getRobot() {
  return this.robot;
}
```

Fig. 3.8: Uso de la anotación `@Ensures` en el valor de retorno de un método

```
public static void pushToStack(
  @Requires("Init") @Ensures("Init") final Stack stack,
  int e
) {
  if (!stack.isFull()) {
    stack.push(e);
  }
}
```

Fig. 3.9: Uso de la anotación `@Ensures` en un parámetro de entrada de un método

Durante nuestra experimentación, notamos que la anotación también puede utilizarse en los parámetros de entrada de un método. Esto no es mencionado por la documentación oficial de la herramienta, por lo que no podemos afirmar que sea el comportamiento

esperado. De todos modos, esta forma de utilizarla sí es mencionada en una de las publicaciones de *JaTyC* [17], y consideramos que es relevante porque permite más flexibilidad a la hora de utilizar objetos protocolados. Esto se debe a que, al combinarla con la anotación `@Requires` en un parámetro de entrada, es posible devolver la referencia lineal del objeto provisto por el cliente, y no una compartida. De esta forma, se evita tener que completar obligatoriamente el protocolo del parámetro dentro del *scope* del método.

Un ejemplo de uso de ambas anotaciones puede observarse en la Figura 3.9, donde se opera con un `Stack`. El método declara que el stack provisto deberá encontrarse en el typestate `Init`, y que lo devolverá en el mismo estado. Para que funcione, *JaTyC* requiere que el parámetro sea `final`, es decir, que la variable no sea reasignada durante la ejecución del método.

3.4. Subtipado

Una de las características más interesantes de la herramienta, similar a lo mencionado en el capítulo 2.1.2, es el soporte para **subtipado**. Es una de las funcionalidades distintivas de *JaTyC* en comparación con su predecesora Mungo, y es crucial dentro del paradigma de los objetos.

Cuando una clase protocolada extiende a otra también protocolada, *JaTyC* se asegura de que el protocolo del subtipo incluya al del supertipo. En otras palabras, toda secuencia válida de llamadas a métodos para el tipo padre debe seguir siéndolo para el tipo hijo. El subtipo tiene solamente la capacidad de **agregar** nuevas secuencias de llamadas a métodos. En cierto sentido, si modelamos el protocolo de un objeto como una máquina de estados finita, el autómata del subtipo debe preservar el comportamiento del autómata del supertipo.

```
1 typestate Iterator {
2   HasNext = {
3     boolean hasNext(): <true: Next,
      false: end>
4   }
5   Next = {
6     boolean hasNext(): <true: Next,
      false: end>,
7     Object next(): HasNext
8   }
9 }
```

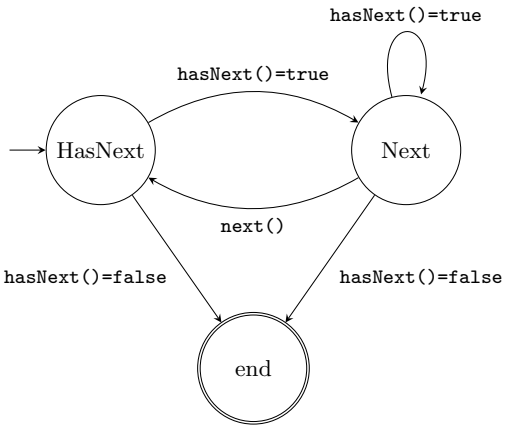


Fig. 3.10: Protocolo `Iterator` y su autómata

Esto es así porque se pretende que el subtipo se "comporte como" el supertipo. En las Figuras 3.10 y 3.11 se pueden ver los protocolos para una interfaz `Iterator` y una clase `LoopIterator` que la implementa. Como indica su nombre, al llegar al final de la colección esta clase reinicia su puntero al inicio, eliminando la necesidad de consultar si hay un elemento siguiente (`hasNext()`). Notemos que las secuencias de métodos válidas para `Iterator` consisten en llamadas a `hasNext()` y una única llamada a `next()` cada vez que haya un elemento siguiente. Observemos que `LoopIterator` permite todo eso y

```
typestate LoopIterator {
  Init = {
    boolean hasNext(): <true: Init,
    false: Init>,
    Object next(): Init,
    drop: end
  }
}
```

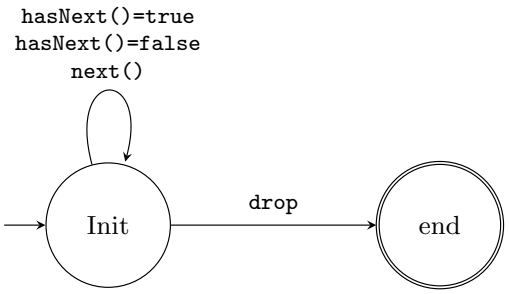
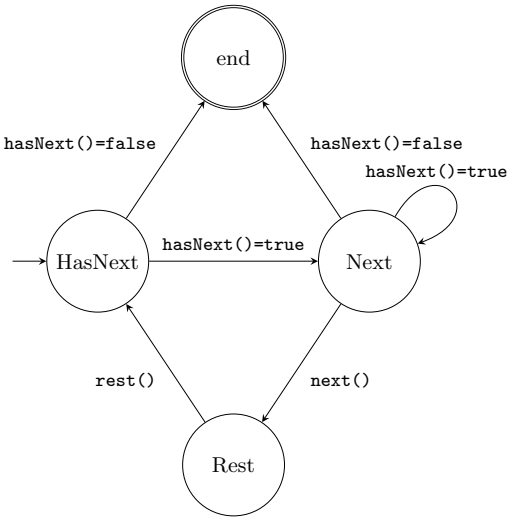


Fig. 3.11: Protocolo `LoopIterator` y su autómata

más, dado que ambos métodos pueden llamarse sin restricciones.

De tener un objeto de tipo `Iterator`, se espera que se comporte como tal y que, por ende, siga su protocolo. Si el protocolo de `LoopIterator` no permitiese *al menos* todas las secuencias de llamadas a métodos de su interfaz, cualquier código cliente podría levantar una excepción al utilizar un objeto `Iterator`. Esto se debe a que, incluso siguiendo el contrato de uso de la interfaz, se podría estar incumpliendo el de la implementación.

```
typestate TiredIterator {
  HasNext = {
    boolean hasNext(): <true: Next,
    false: end>
  }
  Next = {
    boolean hasNext(): <true: Next,
    false: end>,
    Object next(): Rest
  }
  Rest = {
    void rest(): HasNext
  }
}
```




```
TiredIterator.java:6: error: [hasNext] transition(s) in [HasNext] of
    Iterator.protocol are not included in [Rest] of TiredIterator.protocol
public class TiredIterator implements Iterator {
```

Fig. 3.12: Protocolo `TiredIterator`, su autómata y el error que reporta el checker

Un ejemplo de *JaTyC* previniendo esta clase de situaciones puede observarse en la Figura 3.12. La clase `TiredIterator` implementa la misma interfaz, pero su protocolo agrega un nuevo estado `Rest`, que incluye el método `rest()`. Según el protocolo, luego de cada llamada a `next()` se deberá necesariamente "dejar descansar al iterador" llamando a este nuevo método. Dada esta implementación, se puede ver fácilmente que no permite las mismas secuencias de llamadas que su interfaz. Por ejemplo, la secuencia `hasNext()`, `next()` y luego `hasNext()`, permitida por `Iterator`, no puede realizarse siguiendo el protocolo de `TiredIterator`, ya que antes se debe llamar a `rest()`. En la misma figura puede observarse también el error reportado por la herramienta al analizar los protocolos.

Por otro lado, *JaTyC* también permite que una clase no protocolada extienda a otra con protocolo. En este caso, el subtipo sigue el protocolo del supertipo y todos sus nuevos métodos son considerados *anytime*. Finalmente, también es posible que una clase protocolada extienda a una sin protocolo. En esta situación, todos los métodos de la clase padre se consideran *anytime*, y la clase hija funciona como una clase protocolada regular.

Como parte de esta funcionalidad, *JaTyC* también introduce lógica para manejar correctamente la conversión de tipos, o *casting*, tanto implícita como explícita.

3.5. Impacto

Con todas las características mencionadas anteriormente, *JaTyC* consigue determinar en tiempo de compilación que todos los objetos sigan y completen sus protocolos, de tenerlos. En otras palabras, verifica que los métodos de los objetos se llamen en el orden correcto y que los objetos se utilicen hasta el final. Además, mediante sus anotaciones permite detectar potenciales errores de punteros nulos y especificar los typestates esperados para parámetros y valores de retorno de métodos. Finalmente, da soporte al subtipado, permitiendo a los usuarios aplicar herencia entre cualquier clase y manteniendo la correcta verificación de protocolos.

3.6. Limitaciones

Los autores describieron en la documentación oficial [20] algunas limitaciones de la herramienta en las que están trabajando. Consideramos relevante tenerlas en cuenta a la hora de analizar las capacidades de *JaTyC*, dado que condicionan el tipo de programas que pueden verificarse y ayudan a interpretar correctamente algunos resultados obtenidos en nuestros experimentos. A continuación, se detallan las más relevantes para nuestro trabajo.

- No hay soporte para genéricos, los cuales son anulables y de referencia compartida por defecto.
- No hay soporte para excepciones. Esto implica que transiciones entre *typestates* a través de excepciones esperadas [12] no son modelables, y que *JaTyC* no garantiza un estado consistente para un objeto que lanzó una excepción.
- No hay soporte para herencia múltiple.
- Los métodos *anytime* no pueden modificar a los colaboradores internos de un objeto.

3.7. Implementación

En esta sección describiremos la implementación de la versión inicial de *JaTyC*, basada en el trabajo realizado en 2021 por uno de sus autores, João Mota [17]. Reconocemos que esta difiere en algunos aspectos de la versión actual [19], tanto en funcionalidades como en nomenclatura, pero consideramos que se trata de cambios menores y que gran parte de la estructura del código se mantuvo inalterada. Esto nos permitirá entender cómo la herramienta hace el seguimiento del estado de los objetos, cuál es su reticulado y su algoritmo de inferencia de tipos, y cómo detecta incumplimientos de protocolos o de anotaciones adicionales.

3.7.1. Diferencias con la versión actual

Una diferencia notable entre la versión inicial de la herramienta y la actual es que la primera contaba con una anotación adicional, `@State`, que funcionaba de forma idéntica a la actual `@Ensures`: especificaba el *typestate* de un valor de retorno. A su vez, la anotación `@Ensures` existía, pero se utilizaba únicamente para los parámetros de entrada de los métodos, nunca para valores de retorno. A partir de esta diferencia realizamos la prueba mencionada en la sección 3.3.3, en la que descubrimos que el uso de `@Ensures` sobre parámetros sigue funcionando en la versión actual, aunque no figura en la documentación.

Otra diferencia es que la linealidad de la primera versión era más rígida: no se permitía tener más de una referencia utilizable para un mismo objeto. De haber más de una variable apuntando al mismo objeto, solamente la lineal podía llamar métodos, y hacerlo desde las demás producía un error de *JaTyC*. En la versión actual se permite tener más de una referencia a un objeto, pero solo una tiene acceso total al mismo y las demás solo pueden acceder a sus métodos compartidos.

3.7.2. Visión general

La herramienta analiza las clases de un programa en aislamiento, una por una y método a método, en dos fases (Figura 3.13). La primera fase infiere los tipos de las variables y colaboradores internos de todas las clases a verificar. La segunda utiliza estas inferencias para evaluar si existen incompatibilidades de tipos o si se realizaron operaciones inválidas, y de ser así reporta un error. A su vez, verifica que todos los objetos completen su protocolo y que se respete la linealidad.

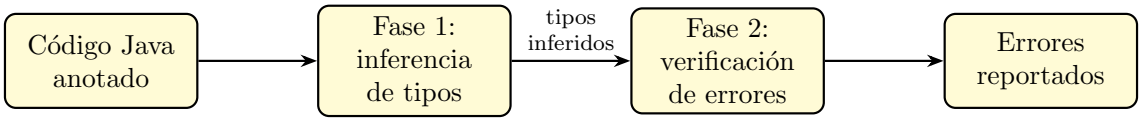


Fig. 3.13: Visión general del análisis de *JaTyC*.

3.7.3. Primera Fase: Inferencia de Tipos

Durante esta fase, *JaTyC* infiere los tipos de las variables, colaboradores internos (los campos de una clase), parámetros y valores de retorno, además de los ya inferidos por el compilador de Java.

Reticulado de tipos

Para realizar la inferencia, la herramienta sigue la estructura y los conjuntos del reticulado de la Figura 3.14.

El tipo ***Unknown*** representa todos los valores posibles y se utiliza cuando se desconoce el tipo del dato. ***Primitive*** incluye todos los tipos primitivos de Java, tales como los enteros o los booleanos. El tipo ***Object*** representa a todos los objetos, exceptuando el valor `null`, que es de tipo ***Null***. ***Moved*** se utiliza con las variables que referencian a objetos que fueron pasados como parámetro a métodos o asignados a otra variable. Este tipo es el equivalente a las referencias compartidas (*shared*) de la herramienta actual, con la diferencia de que las variables de tipo *Moved* no pueden utilizarse en lo absoluto (ni

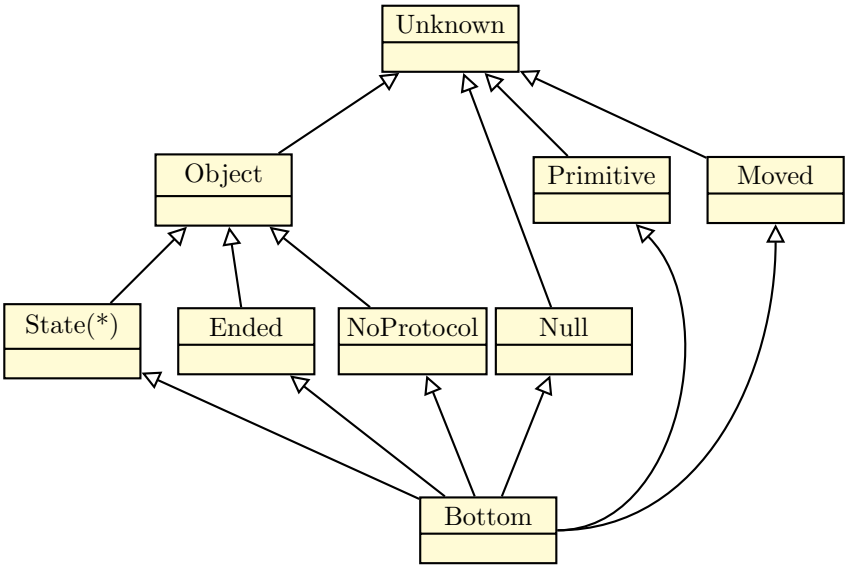


Fig. 3.14: Reticulado de tipos de *JaTyC*.

siquiera con los actuales métodos *anytime*). ***NoProtocol*** incluye a los objetos que no siguen ningún protocolo, y ***Ended*** a los objetos que sí lo siguen y ya lo completaron. Los tipos ***State(*)*** representan a los objetos que siguen un protocolo, aún no lo completaron y se encuentran en un estado específico. Dentro del reticulado hay un tipo por cada *typestate* de cada protocolo. Finalmente, ***Bottom*** es conceptualmente el conjunto vacío, donde se permiten todas las operaciones. Se utiliza para variables fuera de *scope*, resultados de cálculos que generarían errores, etc.

En base a este reticulado, *JaTyC* sigue las siguientes reglas a la hora de refinar el tipo final de una variable:

- El tipo de una variable puede ser cualquiera de los del reticulado o la unión de varios.
- No hay uniones de uniones.
- Si una unión de tipos contiene al tipo *Unknown*, el tipo resultante es simplemente *Unknown*.
- Si una unión de tipos contiene un solo elemento, el tipo resultante es simplemente ese elemento, fuera de la unión.
- Si una unión de tipos no contiene ningún elemento, el tipo resultante es *Bottom*.
- Las uniones no incluyen al tipo *Bottom*, dado que es el conjunto vacío.
- Si el tipo *Object* pertenece a una unión, los tipos *NoProtocol*, *Ended* y *State(*)* no pertenecen a ella, dado que son subtipos.

Class Analysis

Desde una perspectiva de alto nivel, el proceso de inferencia de tipos sucede clase por clase en forma independiente, en lo que Mota denominó *Class Analysis*.

Para cada clase visitada, comienza por corroborar que tenga asociado algún protocolo y, de lo contrario, se le asigna un protocolo trivial. Este consiste en un único *typestate* descartable, en el que todos los métodos de la clase están disponibles y transicionan al mismo estado.

Por otro lado, los métodos se analizan separando los estáticos de los no estáticos. Los primeros se revisan en cualquier orden, mientras que los segundos siguen el orden establecido por el protocolo, comenzando por el constructor. Esto se debe a que la información recopilada durante el análisis de un método se usa en el análisis del siguiente, y al seguir las transiciones del protocolo se evitan las secuencias inválidas de llamadas a métodos. Dado que el análisis de ambos tipos de métodos es muy similar, salvo por pequeñas diferencias, a partir de ahora nos enfocaremos exclusivamente en los métodos no estáticos, que consideramos suficientes para entender el comportamiento general de la herramienta.

Inventarios y fusión. A modo de resumen, a la hora de analizar un método, *JaTyC* recorre todas sus expresiones en orden, de principio a fin, actualizando un inventario con los tipos inferidos para cada variable involucrada, de forma similar a otros analizadores estáticos.

Más específicamente, un **inventario** es un mapeo entre variables y sus tipos inferidos. Por cada expresión del método, la herramienta llama a una **función de transferencia**, que toma dos inventarios y devuelve otros dos: el **inventario *then*** y el **inventario *else***. Si la expresión bajo análisis devuelve un booleano, los inventarios contienen el resultado de analizar la rama verdadera y la falsa, respectivamente. Esto se usa para inferir tipos en presencia de condicionales o ciclos, donde los resultados pueden diferir. Si la expresión analizada no devuelve un booleano, ambos inventarios retornados son iguales. El par de inventarios que recibe la función de transferencia es el que devolvió la función para la expresión anterior.

Para combinar la información de dos inventarios se utiliza la operación de **fusión**, que denotamos FUSIONAR(inv_1, inv_2). El inventario resultante incluye a todas las variables mencionadas en alguno de los dos, y el tipo de cada una es la unión de los tipos que le asignan ambos inventarios. Si una variable solo es mencionada por uno de ellos, conserva su tipo.

Esto busca que el resultado abarque todas las combinaciones de tipos posibles para las variables en juego. Los inventarios *then* y *else* resultantes de analizar la última expresión de un método se fusionan de esta forma para obtener el **inventario de salida**.

El algoritmo. Dadas todas estas definiciones, el algoritmo de *Class Analysis* para los métodos no estáticos es el siguiente:

Se mantienen dos diccionarios a lo largo de la ejecución: uno que relaciona los *typestates* del protocolo con inventarios, T, y otro que relaciona métodos con inventarios, M. Notamos $T(t)$ al inventario asociado al *typestate* t y $M(m)$ al inventario asociado al método m. En estos inventarios se almacenan los tipos inferidos durante el algoritmo, y se utilizan como **inventarios de entrada** para el análisis de cada método. Sobre esto profundizaremos más adelante.

Al comienzo del algoritmo, todos los métodos y *typestates* de los diccionarios se inicializan con el inventario vacío $inv_\emptyset$, que por defecto asigna *Bottom* a todas las variables. Luego, *JaTyC* infiere los tipos iniciales de los colaboradores internos del objeto analizando el método constructor, y asocia el inventario de salida resultante al *typestate* inicial del protocolo, t_0.

Algorithm 1 Análisis de una clase con protocolo en *JaTyC*

Entrada: Clase con protocolo P (*typestate* inicial t_0) y constructor c
Salida: Diccionarios T (*typestates* $\rightarrow$ inventarios) y M (métodos $\rightarrow$ inventarios)

```
1: T ← diccionario de typestate a inventario, con inv_∅ por defecto
2: M ← diccionario de método a inventario, con inv_∅ por defecto
3: T(t_0) ← ANALIZAR(c, inv_∅)
4: Q ← [ t_0 ]
5: while Q no está vacía do
6:     t ← Q.DESENCOLAR()
7:     for método m alcanzable desde t en P do
8:         inv_in ← FUSIONAR(T(t), M(m))
9:         if inv_in = M(m) then
10:            continue                ▷ m ya fue analizado con este inventario
11:        end if
12:        M(m) ← inv_in
13:        inv_out ← ANALIZAR(m, M(m))
14:        t_d ← typestate destino de llamar a m desde t en P
15:        inv_new ← FUSIONAR(inv_out, T(t_d))
16:        if inv_new = T(t_d) then
17:            continue        ▷ No se aprendió nada nuevo al analizar m con M(m)
18:        end if
19:        T(t_d) ← inv_new
20:        Q.ENCOLAR(t_d)
21:    end for
22: end while
```

Por otro lado, se mantiene una cola Q que contiene los *typestates* a analizar. Esta se utiliza para realizar una especie de *BFS* sobre el grafo del protocolo de la clase, de modo de analizar solo cadenas de métodos válidas. Q se inicializa conteniendo solamente el *typestate* inicial t_0.

El algoritmo comienza entonces un ciclo en el que, en cada iteración, se desencola el próximo *typestate* t y se lo analiza, hasta que la cola queda vacía. En el cuerpo del ciclo, la herramienta hace lo siguiente:

1. Recorre cada uno de los métodos m alcanzables desde t.

2. Para cada uno, fusiona $T(t)$ con $M(m)$. Si el resultado es idéntico a $M(m)$, se omite el análisis del método. En otras palabras, se considera que m ya fue analizado con ese mismo inventario y que, por ende, no se aprenderá nada nuevo.

3. De lo contrario, se actualiza $M(m)$ con el resultado de la fusión y se lo utiliza como inventario de entrada para analizar m.

4. Denotando como t_d al *typestate* destino luego de llamar a m en el protocolo, se fusiona el inventario de salida del análisis de m con $T(t_d)$.

5. Si el resultado de la fusión es igual a $T(t_d)$, se continúa sin actualizar nada, ya que se entiende que no se aprendió nada nuevo al analizar m con $M(m)$.

6. De lo contrario, se actualiza $T(t_d)$ con el resultado de la fusión y el *typestate* se encola para analizarlo a futuro.

Como mencionamos previamente, el ciclo termina cuando no quedan *typestates* en la cola. Mota garantiza su finalización dado que el número de estados de un protocolo es finito y que no se reencolan estados de los cuales no se aprendió nada nuevo luego de analizarlos. Dicho de otra forma, un *typestate* solo se reencola cuando su inventario cambia, y como la fusión únicamente puede ampliar los tipos de cada variable (nunca los reduce), y tanto el reticulado como el conjunto de variables de una clase son finitos, cada inventario puede cambiar solo una cantidad finita de veces.

Análisis de Métodos

A lo largo de la ejecución del algoritmo, la función Analizar(m, inv) se utiliza para inferir tipos en cada uno de los métodos. Como mencionamos anteriormente, esta recorre en orden todas las expresiones, una por una, llamando a la función de transferencia y actualizando los inventarios *then* y *else*. A continuación exponemos cómo funciona internamente.

Introducción de tipos. Antes de comenzar con la primera expresión, Analizar(m, inv) construye un inventario inicial, que será la entrada de la primera llamada a la función de transferencia. Este es el resultado de la fusión de dos inventarios. El primero es el provisto por el propio *Class Analysis* (denotado $M(m)$ en el Algoritmo 1). El segundo se obtiene a través de la **introducción de tipos**, un paso inicial del análisis de un método que asigna tipos iniciales a parámetros y colaboradores internos según cómo y dónde se declaran, observando el tipo de Java mencionado en la expresión. La Tabla 3.1 resume los tipos asignados en cada caso.

Para algunos tipos de Java, esta introducción es directa. A todo `java.lang.Object` encontrado se le asigna el tipo *Object*. A los primitivos, como enteros o booleanos, se les asigna el tipo *Primitive*. A todo valor `null` encontrado se le asigna el tipo *Null*. A las variables y colaboradores internos de un tipo sin protocolo se les asigna el tipo *NoProtocol*.

La introducción de tipos para variables y colaboradores internos cuyo tipo es una clase con protocolo es distinta según el caso:

- **Valores de retorno del método:** se les asigna el tipo correspondiente al estado especificado en la anotación `@State` (equivalente a la actual `@Ensures`). Si no se cuenta con una anotación, se les asigna la unión de los tipos correspondientes a todos los estados de su protocolo excepto el reservado `end`, representando que pueden encontrarse en cualquier estado posible. Esto ya no es así en la versión actual: si no se utiliza la anotación `@Ensures`, se interpreta que la referencia devuelta es compartida.

- **Parámetros del método:** funciona de la misma forma, pero con los *typestates* especificados por `@Requires`. Similarmente, en las versiones más nuevas de *JaTyC* esto ya no funciona así: los parámetros sin la anotación se interpretan como compartidos.

- **Colaboradores internos públicos:** se les asigna el tipo *Unknown*, dado que la herramienta no puede garantizar el estado de objetos que pueden ser alterados desde cualquier otra clase.

Elemento	**Tipo inicial**
`java.lang.Object`	*Object*
Tipos primitivos (enteros, booleanos, etc.)	*Primitive*
Valor `null`	*Null*
Variables y colaboradores internos de un tipo sin protocolo	*NoProtocol*
Elementos de un tipo con protocolo	
Valores de retorno	Tipo del estado especificado en `@State`. Sin anotación, unión de los tipos de todos los estados del protocolo excepto el reservado `end`
Parámetros	Ídem, con los *typestates* especificados en `@Requires`
Colaboradores internos públicos	*Unknown*
Colaboradores internos privados y variables locales	Unión de los tipos de todos sus *typestates* posibles

Tab. 3.1: Introducción de tipos según el elemento declarado.

- **Colaboradores internos privados y variables locales del método:** se les asigna la unión de los tipos correspondientes a todos sus *typestates* posibles. Esto representa que, salvo que se infiera algo distinto durante el análisis del método, pueden encontrarse en cualquier estado.

Finalmente, en cualquiera de los casos anteriores, si se utiliza la anotación `@Nullable` antes de un tipo de Java, el tipo correspondiente incluye inicialmente a *Null*, que luego puede refinarse durante el proceso de inferencia.

Reglas de inferencia. Luego de obtener el inventario inicial, ANALIZAR(m, inv) recorre las expresiones una por una actualizando los tipos inferidos. Lo hace bajo ciertas reglas de inferencia, que cambian según el tipo de expresión a analizar:

- **Instanciación** `Obj x = new Obj()`: se le asigna a `x` el tipo correspondiente al *typestate* inicial de `Obj`, de tener protocolo. De lo contrario, se le asigna el tipo *NoProtocol*.
- **Asignación** `x = y`: a `x` se le asigna el tipo de `y`. Si `y` es de tipo *Object* o de alguno de sus subtipos, pasa a ser *Moved*; de lo contrario, permanece sin cambios. Esto le quita la capacidad de operar sobre el objeto al que referenciaba y preserva la linealidad. En la versión actual de *JaTyC*, `y` pasaría a ser una variable compartida.
- **Retorno** `return x`: el tipo de `x` permanece sin cambios, a no ser que sea un objeto con protocolo, en cuyo caso pasa a ser *Moved*. Aunque luego de esta expresión ya se sale del *scope* del método y parecería insignificante cambiar su tipo en ese punto, se hace por si el objeto devuelto es un colaborador interno, denotando que esta referencia ya no puede utilizarse más adelante.

- **Llamada a un método `x.m()`**: el tipo inferido para `x` cambia según su tipo inicial, como resume la Tabla 3.2. Por otro lado, el tipo del resultado de la llamada `x.m()` es el mismo que el del valor retornado por `m`. A su vez, a los argumentos se les asigna *Moved*, de forma similar a como la herramienta actual los convierte en referencias compartidas. No obstante, si el método llamado los anota con `@Ensures`, se les asigna el tipo correspondiente al *typestate* especificado.

Tipo inicial de `x`	**Tipo de `x` luego de `x.m()`**
NoProtocol	*NoProtocol*
Ended, *Moved*, *Null* o *Primitive*	*Bottom* (operación inválida)
Objeto con protocolo (*State(*)*)	Tipo del estado destino de `m`, o *Bottom* si `m` no estaba disponible en su estado actual
Unknown u *Object*	Sin cambios, dado que no se puede saber el tipo resultante luego de llamar a `m`
Bottom	*Bottom*
Unión de tipos	Se aplican las reglas anteriores a cada miembro de la unión

Tab. 3.2: Tipo inferido para el receptor `x` luego de la llamada `x.m()`.

Control de flujo. Para razonar sobre el flujo de ejecución, los métodos se recorren a través de su *control flow graph*, construido por el *Checker Framework*, del cual hablaremos en la sección 3.7.5. Como vimos, el resultado de analizar cada expresión es un par de inventarios *then* y *else*, que describen los tipos de las variables cuando la expresión evalúa a verdadero o falso. Esto permite refinar el tipo de un objeto en cada rama de un condicional o ciclo.

Para ilustrarlo, consideremos el protocolo mostrado anteriormente para un iterador básico de la Figura 3.10. En el estado *HasNext*, el método `hasNext()` transiciona a *Next* si devuelve verdadero y a `end` si devuelve falso. En el estado *Next*, el método `next()` vuelve a *HasNext*.

Si `hasNext()` se llama en la condición de un `if`, el inventario *then* contiene al iterador en el estado *Next* y el inventario *else* lo contiene como *Ended*. Cada uno de ellos se propaga a su rama respectiva, lo que permite conocer con más precisión el estado del iterador (Figura 3.15).

```
1 Iterator it = new Iterator();
2 // it: State "HasNext"
3 if (it.hasNext()) {
4   // it: State "Next"
5   it.next();
6 } else {
7   // it: Ended
8 }
```

Fig. 3.15: Refinamiento de tipos en un `if-else`.

El mismo razonamiento se aplica cuando la llamada se realiza en otras expresiones de control de flujo, tales como un `while`. La diferencia es que el inventario *then* se propaga al cuerpo del ciclo, y el inventario *else* al punto del programa al que se llega al salir de este (Figura 3.16).

```
1 Iterator it = new Iterator();
2 // it: State "HasNext"
3 while (it.hasNext()) {
4   // it: State "Next"
5   it.next();
6   // it: State "HasNext"
7 }
8 // it: Ended
```

Fig. 3.16: Refinamiento de tipos en un `while`.

3.7.4. Segunda Fase: Verificación de Errores

En esta fase se utiliza toda la información de tipos inferidos para buscar potenciales errores. Estos incluyen desde incompatibilidades de tipos y potenciales errores de punteros nulos hasta protocolos incompletos.

La siguiente es una lista con las principales verificaciones que realiza *JaTyC* en esta fase:

- Para asignaciones `x = y`, se asegura que `y` sea del tipo esperado por `x` o de un subtipo. A su vez, si `x` es un objeto con protocolo, verifica que su tipo sea *Ended*, *Moved* o un tipo correspondiente a un *typestate* descartable. De no ser así, se perdería la única referencia lineal al objeto y este no podría finalizar su protocolo.
- Para expresiones de retorno `return x`, verifica que el tipo de `x` se corresponda con el especificado por el método.
- Para llamadas a métodos `x.m()`, el tipo del objeto receptor `x`, de tener protocolo, debe corresponder a un *typestate* habilitado para llamar a `m`, y no a otro como *Moved*, *Null* o *Ended*. Los argumentos también deben ser subtipos de los esperados por `m`. Además, si `m` devuelve un objeto con protocolo aún no finalizado, verifica que este se asigne a una variable para que pueda completarlo, o que esté en un *typestate* descartable.
- Para toda operación sobre un valor, como la llamada a un método, verifica que su tipo no pueda ser *Null*, o que haya sido refinado para excluirlo. De lo contrario, se reporta un posible error de puntero nulo.
- Revisa todas las variables mencionadas en los inventarios de salida de los métodos, junto con sus tipos inferidos. Para toda variable que referencie a un objeto con protocolo, se revisa su origen. Si es un parámetro del método anotado con `@Ensures`, debe tener al finalizar el método el tipo correspondiente a la anotación. Los parámetros no anotados y las variables locales deben tener tipo *Ended*, *Moved* o uno correspondiente a un estado descartable.

- Revisa los inventarios asociados a *typestates* resultantes del *Class Analysis* (los del diccionario T) correspondientes a estados finales del protocolo, tanto `end` como descartables. En ellos, verifica que todos los colaboradores internos de la clase que referencien a un objeto con protocolo lo hayan completado o hayan delegado la referencia lineal, teniendo el tipo *Ended*, *Moved* o uno correspondiente a un *typestate* descartable.
- Revisa que no se envíen objetos con protocolo no finalizado como parámetros a métodos no analizables, por ejemplo código de bibliotecas de terceros. Esto se debe a que no se puede garantizar qué sucederá con el objeto durante la llamada ni si finalizará su protocolo allí.

3.7.5. Checker Framework

A diferencia de su predecesora Mungo, creada con el framework JastAdd [13], *JaTyC* fue desarrollada con el **Checker Framework**. Como mencionamos en la sección 2.3.2, este le permite a los desarrolladores incluir varios *checkers* en sus proyectos, los cuales realizan diferentes tipos de verificaciones en tiempo de compilación. El Checker Framework extiende el sistema de tipado nativo de Java y, apoyándose en las interfaces del módulo de compilación de Java [22], provee clases e interfaces simples y accesibles para analizar código. *JaTyC* las utiliza para hacer sus verificaciones de protocolos, y consideramos interesante dedicar un apartado a recorrer lo que consideramos sus principales usos. Todas las clases e interfaces mencionadas en las próximas secciones pueden encontrarse en la documentación oficial del framework [9] y en el código oficial de *JaTyC* [19].

SourceChecker

El `SourceChecker` es una de las principales clases ofrecidas por el Checker Framework. Es el punto de entrada al framework y hace de interfaz hacia la API de procesamiento de anotaciones (*annotation processing*) del compilador de Java. También funciona como objeto *factory* de otras clases interesantes del framework, las cuales mencionaremos más adelante.

La clase se inicializa con varios valores y colaboradores internos por defecto para ser "*plug and play*", útiles para la mayoría de los *checkers* que se quieran construir. De todos modos, muchos de estos valores pueden sobrescribirse para modificar y flexibilizar el comportamiento de un *checker*, ventaja de la cual *JaTyC* hace uso extensivo.

JaTyC tiene su propia subclase de `SourceChecker`, el `JavaTypestateChecker`. Principalmente, esta inicializa algunas clases de utilidades, agrega soporte para imprimir mensajes de alerta y de error y, por último, sobrescribe el método `createSourceVisitor()` para devolver un `SourceVisitor` personalizado para la herramienta: el `CFVisitor`.

SourceVisitor

Otra de las clases importantes del Checker Framework es el `SourceVisitor`, cuyo principal propósito es ser un *visitor* para **Árboles de Sintaxis Abstracta** (*AST*s). Sus instancias saben recorrer el *AST* de cada archivo fuente y ofrecen la funcionalidad para analizar su contenido. Si bien la clase en sí no realiza verificaciones de código, el framework ofrece una subclase, `BaseTypeVisitor`, que reporta errores cuando se incumple el sistema de tipos en ciertos casos, como asignaciones o llamadas a métodos. En otras

palabras, el `SourceVisitor` tiene la lógica para recorrer los *AST*s, y sus subclases la aprovechan para agregar las verificaciones. Los árboles que recorre son provistos por la API del módulo de compilación de Java [22], y la clase extiende una de las clases de esa API, `TreePathScanner`, que ya contiene la lógica para recorrer un *AST* hacia sus nodos hijos y mantiene el camino hasta la raíz (`TreePath`), lo que permite acceder también a los nodos padres.

Como mencionamos en la sección anterior, el `SourceChecker` se encarga de proveer su propio `SourceVisitor`. En el caso de *JaTyC*, el `JavaTypestateChecker` inicializa su propia subclase, el `CFVisitor`. A diferencia del `BaseTypeVisitor`, que verifica el código a la par que lo recorre, el `CFVisitor` realiza su análisis en dos fases.

La primera fase sí se parece al comportamiento habitual de las subclases del `SourceVisitor`: aprovecha su lógica de recorrido de *AST*s, pero no realiza ninguna tarea relacionada con el *Class Analysis* ni con la verificación de *typestates*. Su objetivo es construir la representación interna de *JaTyC*. Por ejemplo, el `CFVisitor` sobrescribe el método `visitClass()` para que, por cada clase visitada, esta se traduzca a su representación interna (`ClassDeclAndCompanion`), se guarde en un mapa indexado por su nombre y se encole para ser analizada en la segunda fase.

Otra función de la primera fase es validar las anotaciones de *JaTyC* presentes en los archivos, mediante el método `visitAnnotation()`. Observando los nodos de los árboles provistos por el compilador, se asegura que `@Requires` solo aparezca en parámetros de métodos, `@Ensures` en parámetros o tipos de retorno, y `@Typestate` en clases o interfaces. Además, se verifica que los estados mencionados en estas anotaciones existan en el protocolo del objeto que especifican.

Para realizar las traducciones, el `CFVisitor` delega en otra clase importante del *checker*: el `CFAdapter`. Esta se encarga de convertir los modelos provistos por el Checker Framework en las representaciones internas de *JaTyC*. Entre las más importantes, para cada método (así como para cada inicializador y lambda), construye con el framework su grafo de control de flujo (`ControlFlowGraph`) y lo convierte en un `SimpleCFG`, una versión simplificada que la herramienta sabe procesar. A su vez, cada nodo (`Node`) de esos grafos se traduce a una o más expresiones de código internas (`CodeExpr`), que describen únicamente lo necesario para realizar la verificación.

La segunda fase del `CFVisitor` sucede una vez que el Checker Framework termina de visitar el código y la representación interna de *JaTyC* ya fue construida. Durante esta fase, el análisis deja de depender del framework y toda la lógica de verificación de protocolos y anotaciones adicionales es propia de la herramienta: es aquí donde se realiza todo lo mencionado en la sección de implementación 3.7, con la inferencia de tipos y la verificación de errores. El framework solo vuelve a utilizarse para reportar los errores y advertencias en la posición correcta del código fuente.

Clases Utilitarias

JaTyC también hace uso de varias clases utilitarias del framework. En particular, `ElementUtils`, `TypesUtils`, `AnnotationUtils` y `TreeUtils`.

`ElementUtils` sabe analizar `Element`s (elementos). Esta es una interfaz del compilador de Java que representa, semánticamente, la declaración de una construcción del lenguaje durante la compilación. El manual oficial del Checker Framework [10] describe que existe una implementación de esta interfaz para cada tipo de declaración de Java: por ejemplo, los `TypeElement` para las clases e interfaces, los `ExecutableElement` para los métodos y

constructores, o los `VariableElement` para los campos, las variables locales y los parámetros de métodos. *JaTyC* utiliza esta clase, por ejemplo, para obtener la clase que define a un método o para verificar si los miembros de una clase son estáticos o no.

`TypesUtils` sirve para razonar sobre los `TypeMirror`s. Esta también es una interfaz del compilador de Java, y representa los tipos de datos, tales como los primitivos, las clases, las interfaces, etc. *JaTyC* utiliza esta clase para determinar el tipo exacto de las variables, como por ejemplo si son objetos o valores inmutables, o para analizar la relación jerárquica entre diferentes tipos.

`AnnotationUtils` se utiliza para trabajar con instancias de `AnnotationMirror`, que son simplemente representaciones de las anotaciones de Java. La herramienta usa principalmente esta clase para obtener el nombre de las anotaciones.

Finalmente, `TreeUtils` es una clase que analiza los nodos de los árboles de sintaxis abstracta del compilador de Java. Estos nodos son diferentes clases que implementan la interfaz `Tree` y representan unidades sintácticas (y no semánticas) del código, tales como declaraciones de métodos, condicionales, ciclos `while` o `for`, etc. *JaTyC* usa esta clase para extraer `Element`s a partir de los `Tree`s, es decir, para razonar sobre la semántica de un programa a partir de su sintaxis.

4. EXPERIMENTACIÓN

A lo largo de este capítulo se recopilará la experimentación realizada con *JaTyC*. Recordemos que el propósito principal es analizar las capacidades y limitaciones de la herramienta, y a su vez validar que se comporte correctamente en casos interesantes. También buscamos entender mejor el potencial de la verificación de protocolos de objetos con typestates en general.

Las pruebas se realizaron sobre pequeños extractos de código, compuestos por diferentes clases, interfaces, protocolos y clientes. Estos se analizaron con la herramienta y se anotó el comportamiento observado. Cada sección del capítulo tratará un aspecto diferente de *JaTyC* o de la verificación de protocolos de objetos en general, e incluirá algunas de las pruebas más relevantes.

4.1. Protocolos Compuestos

Nos interesa saber en qué medida *JaTyC* tiene en cuenta el estado de los colaboradores internos de un objeto protocolado. La herramienta parece interpretar un typestate simplemente como un conjunto de métodos habilitados para una instancia y sus transiciones hacia otros typestates. El estado interno de un objeto pasa a ser secundario para la herramienta, aunque entiende que solo debe ser modificado por métodos protocolados y no por los *anytime*. En este sentido, el protocolo y el uso debido de un objeto están dictados solamente por las secuencias válidas de sus métodos.

```
import jatyc.lib.*;

@Typestate("RobotProtocol")
public class Robot {
  private int x;
  private int y;

  public Robot() {
    this.x = 0;
    this.y = 0;
  }

  public void moveLeft() {
    this.x--;
    if (x < 0) {
      throw new RuntimeException("I
    fell off the map!");
    }
  }

  public void moveRight() {...}
  public void moveUp() {...}
  public void moveDown() {...}
}
```

```
typestate RobotProtocol {
  TopLeft = {
    void moveRight(): TopRight,
    void moveDown(): BotLeft,
    drop: end
  }
  BotLeft = {
    void moveRight(): BotRight,
    void moveUp(): TopLeft,
    drop: end
  }
  TopRight = {
    void moveLeft(): TopLeft,
    void moveDown(): BotRight,
    drop: end
  }
  BotRight = {
    void moveLeft(): BotLeft,
    void moveUp(): TopRight,
    drop: end
  }
}
```

Fig. 4.1: Código y protocolo para un `Robot`, capaz de moverse sobre un mapa de tamaño 2×2.

Lo que queremos aprender es si, a pesar de esto, la herramienta es capaz de razonar sobre "protocolos compuestos", donde un objeto protocolado tiene de colaborador interno a otro objeto protocolado. ¿Puede el comportamiento de un método de un objeto padre depender del typestate del colaborador? Es decir, ¿puede el protocolo del padre ser uno compuesto, de modo que, dependiendo del typestate de un colaborador, llamadas a un mismo método produzcan resultados distintos (como excepciones en ciertas ocasiones)? Intentamos entender cómo la herramienta analiza estos casos, donde determinar el comportamiento correcto de un objeto parecería exceder su propio alcance.

Nuestros experimentos buscaban romper el protocolo de un colaborador interno respetando el del objeto padre. En otras palabras, nos interesaba saber si, desde el punto

```java
import jatyc.lib.*;

@Typestate("RobotControllerProtocol")
public class RobotController {
  private Robot robot;

  public RobotController() {
    this.robot = new Robot();
  }

  public boolean canMoveRobot() {
    return true;
  }

  public void move(String direction) {
    switch (direction) {
      case "up":
        this.robot.moveUp(); break;
      case "down":
        this.robot.moveDown(); break;
      case "left":
        this.robot.moveLeft(); break;
      case "right":
        this.robot.moveRight(); break;
      default:
        throw new RuntimeException("Invalid movement direction");
    }
  }
}
```

```
typestate RobotControllerProtocol {
  CanMove = {
    boolean canMoveRobot(): <true: Move, false: CanMove>,
    drop: end
  }
  Move = {
    void move(String): CanMove
  }
}
```

Fig. 4.2: Código y protocolo para un `RobotController`, el cual mueve a un robot en las direcciones cardinales.

de vista de un cliente, se podían provocar comportamientos inesperados en un objeto a pesar de utilizarlo correctamente, causados por su estado interno. Buscábamos entender si *JaTyC* reaccionaría ante estas situaciones y si reportaría un error durante la compilación.

El código y el protocolo de la clase `Robot` pueden observarse en la Figura 4.1. El robot se mueve en un mapa de tamaño 2×2, una unidad de distancia a la vez, con los métodos `moveLeft()`, `moveRight()`, `moveUp()` y `moveDown()`. Comienza en la posición inicial, arriba a la izquierda, y si se mueve a una posición fuera del mapa lanzará una excepción, simulando "caerse del mapa". Su protocolo refleja este comportamiento, con un typestate para cada posición posible del robot y solo los movimientos permitidos, de modo que nunca se caiga.

Un primer experimento con este robot se realizó con el controlador de la Figura 4.2. El `RobotController` tiene un `Robot` como colaborador interno y simplemente toma instrucciones de un cliente para moverlo. Su protocolo es pequeño: cuenta con un método de control inicial `canMoveRobot()`, que habilita al método `move(String direction)`, el cual mueve al robot en la dirección deseada.

```
public class Main {
  public static void main(String args[]) throws Exception {
    RobotController controller = new RobotController();

    if (controller.canMoveRobot()) {
      controller.move("left");
    }
  }
}
```

```
RobotController.java:18: error: Cannot call [moveUp] on State{Robot,
    TopLeft}
        this.robot.moveUp();
                   ^
RobotController.java:24: error: Cannot call [moveLeft] on State{Robot,
    TopLeft}
        this.robot.moveLeft();
                   ^
2 errors
```

Fig. 4.3: Errores reportados por *JaTyC* al analizar el código del controlador en la Figura 4.2 y su respectivo código cliente `Main`.

Por su parte, el código cliente de la Figura 4.3 sigue el protocolo del controlador pero rompe el del robot. Esto se debe a que mueve al robot hacia la izquierda desde la posición inicial, con lo cual cae fuera del mapa. Lo que busca el experimento es generar una situación en la que el cliente, a pesar de usar correctamente el controlador, quebrante sin saberlo el protocolo del robot.

El error devuelto por la herramienta muestra que detecta claramente el incumplimiento del protocolo del robot. En concreto, indica que no pueden llamarse `moveUp` ni `moveLeft` sobre un robot en el typestate `TopLeft`, ambas llamadas dentro del `RobotController`. Lo interesante es que no culpa al código cliente, cuyas llamadas al controlador harían caer al robot fuera del mapa, sino al método `move(String direction)` del `RobotController`. Esto tiene sentido, dado que dicho método no controla el *input* del cliente y es capaz de

mover al robot fuera del mapa: no utiliza métodos de control para asegurarse de que la dirección del movimiento sea segura.

Una conclusión que puede sacarse de esto es que el verdadero cliente del robot no es el código en `Main`, sino el controlador mismo. *JaTyC* reportó el posible incumplimiento del protocolo desde el `RobotController` porque es allí donde se utiliza el robot. Por ende, no analiza ambos protocolos como uno compuesto desde `Main`, sino que verifica por separado a cada cliente de un objeto protocolado.

Una variación del experimento que muestra el mismo comportamiento puede verse en la Figura 4.4. Mientras que el `Robot` no tiene cambios, el `RobotController` ahora cuenta con un método `run()`, que hace que el robot dé una vuelta al mapa en sentido horario, y con `moveRight()`, que mueve al robot una posición hacia la derecha. Notemos que la única posición segura del robot desde la cual puede llamarse a `run()` es la inicial.

```
import jatyc.lib.*;

@Typestate("RobotControllerProtocol")
public class RobotController {
    private Robot robot;

  public RobotController() {
    this.robot = new Robot();
  }

  public boolean canMoveRobot() {
    return true;
  }

  public void run() {
    this.robot.moveRight();
    this.robot.moveDown();
    this.robot.moveLeft();
    this.robot.moveUp();
  }

  public void moveRight() {
    this.robot.moveRight();
  }
}
```

```
typestate RobotControllerProtocol {
  CanMove = {
    boolean canMoveRobot(): <true: Move, false: CanMove>,
    drop: end
  }
  Move = {
    void run(): CanMove,
    void moveRight(): CanMove
  }
}
```

Fig. 4.4: Código y protocolo de la clase `RobotController`, la cual incluye los nuevos métodos `run()` y `moveRight()`.

JaTyC analizó el código con dos clientes diferentes. El primero solamente realizaba

una llamada a `canMoveRobot()` y otra a `run()`, respetando los protocolos tanto del robot como del controlador. El segundo (Figura 4.5) llama a `run()`, luego a `moveRight()` y finalmente a `run()`, todas precedidas por llamadas a `canMoveRobot()`. Comenzando en la posición inicial del mapa de 2×2, arriba a la izquierda, la primera llamada a `run()` deja al robot en la misma posición. Al llamar a `moveRight()`, el robot pasa a estar arriba a la derecha. Finalmente, la última llamada a `run()` hace que el robot se caiga del mapa, dado que comienza con un movimiento a la derecha, y así rompe su protocolo.

```
public class Main {
  public static void main(String args[]) throws Exception {
    RobotController controller = new RobotController();

    if (controller.canMoveRobot()) {controller.run();}
    if (controller.canMoveRobot()) {controller.moveRight();}
    if (controller.canMoveRobot()) {controller.run();}
  }
}
```

```
RobotController.java:23: error: Cannot call [moveRight] on State{Robot,
    TopLeft} | State{Robot, TopRight}
                this.robot.moveRight();
                           ^
RobotController.java:16: error: Cannot call [moveRight] on State{Robot,
    TopLeft} | State{Robot, TopRight}
                this.robot.moveRight();
                           ^
2 errors
```

Fig. 4.5: Errores reportados por *JaTyC* al analizar el código del controlador en la Figura 4.4 y su respectivo código cliente `Main`.

Para ambos códigos clientes, la herramienta reporta los mismos errores de la Figura 4.5: no se puede mover al robot hacia la derecha dado que se encuentra en los posibles typestates `TopLeft` y `TopRight`, para ambos métodos `run()` y `moveRight()` del controlador. En otras palabras, *JaTyC* detecta que el robot puede estar en cualquiera de esos dos estados, y para el caso del typestate `TopRight` se caería del mapa de moverse a la derecha nuevamente.

En particular, el primer cliente, que nunca hace caer al robot, recibe los mismos errores que el segundo, que sí lo hace. Estos errores señalan solamente el uso indebido del robot por parte del controlador, donde diferentes combinaciones de llamadas a métodos provocan que caiga del mapa. Esto es consistente con lo observado en el primer experimento: para *JaTyC* los protocolos no son compuestos, y se analizan por separado el uso del controlador por parte del código cliente y el uso del robot por parte del controlador.

En resumen, ambos experimentos sugieren que *JaTyC* realiza una verificación modular: se analiza el uso de los objetos protocolados en cada clase por separado, pero el protocolo de un objeto padre parece no depender del de sus colaboradores internos. Una consecuencia es que la herramienta rechaza al controlador independientemente de cómo lo use el cliente, lo cual es conservador y puede reportar errores (falsos positivos) en programas que, vistos en conjunto, nunca harían caer al robot. Para que el controlador sea aceptado, debería garantizar por sí mismo que nunca viola el protocolo del robot, por ejemplo validando la dirección mediante métodos de control o reflejando en su propio protocolo las

posiciones posibles del robot. Esto puede verse como un enfoque por *JaTyC* en la división de responsabilidades y encapsulamiento, donde cada objeto es el encargado de asegurarse de funcionar correctamente y cumplir con protocolos. También puede entenderse como una decisión de diseño que prioriza la simplicidad del análisis a costa de expresividad.

4.2. Correctitud vs. Orden de Métodos

En general, el análisis de *object protocols* se limita a verificar el orden correcto de las llamadas a métodos. Mientras que *JaTyC* puede hacer mucho más que eso, como detectar posibles errores de puntero nulo o especificar *typestates* esperados para parámetros o valores de retorno, nos preguntamos en qué medida tiene en cuenta el contenido de los métodos, y no solo lo que el protocolo predica sobre ellos. En principio, parecería no juzgar la **correctitud** de los métodos de un objeto protocolado, por lo que pueden ocurrir excepciones inesperadas incluso si se respeta su contrato. También puede suceder lo contrario: que un código sea correcto bajo cierta especificación y que *JaTyC* reporte potenciales errores porque no se sigue algún protocolo. En ese sentido, es interesante indagar cómo está limitada la herramienta en este aspecto, y cómo podemos utilizarla para que nuestro código se alinee con su especificación de correctitud.

```
import jatyc.lib.*;

@Typestate("StackProtocol")
public class Stack {
  private int[] stack;
  private int top;

  public Stack(int size) {
    this.stack = new int[size];
    this.top = 0;
  }

  public boolean isEmpty() {
    return this.top == 0;
  }

  public boolean isFull() {
    return false;
  }

  public void push(int newValue) {
    this.stack[this.top++] =
    newValue;
  }

  public int pop() {
    return this.stack[--this.top];
  }
}
```

```
typestate StackProtocol {
  Init = {
    boolean isEmpty(): <true: Init,
    false: CanPop>,
    boolean isFull(): <true: Init,
    false: CanPush>,
    drop: end
  }
  CanPush = {
    boolean isEmpty(): <true: Init,
    false: CanPop>,
    boolean isFull(): <true: Init,
    false: CanPush>,
    void push(int): Init,
    drop: end
  }
  CanPop = {
    boolean isFull(): <true: Init,
    false: CanPush>,
    boolean isEmpty(): <true: Init,
    false: CanPop>,
    int pop(): Init,
    drop: end
  }
}
```

Fig. 4.6: Código y protocolo de una clase `Stack`. La misma tiene capacidad limitada y no controla correctamente si está llena.

En la Figura 4.6 puede verse una de las pruebas que realizamos al respecto, con una

clase `Stack` implementada incorrectamente. Según su protocolo, se pueden apilar y desapilar elementos mientras la pila no esté llena ni vacía, respectivamente. Por ejemplo, solo se puede llamar a `push()` si `isFull()` retorna falso. En esta implementación, el `Stack` tiene una capacidad no expandible, definida mediante un parámetro `size` en el constructor. Sin embargo, el método `isFull()` siempre devuelve falso, con lo cual permite apilar elementos hasta sobrepasar el tamaño de la pila, lo que provoca una excepción.

```
1 public class Main {
2   public static void main(String args[]) throws Exception {
3     Stack stack = new Stack(1);
4     while (!stack.isFull()) {
5       stack.push(5);
6     }
7   }
8 }
```

```
1 Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index
    -1 out of bounds for length 1
2         at Stack.push(Stack.java:21)
3         at Main.main(Main.java:5)
```

Fig. 4.7: Código cliente del `Stack`, el cual intenta apilar más elementos de los que caben en la pila, y el error al ejecutarlo.

El código cliente de la Figura 4.7 usa la pila siguiendo su protocolo, pero apila más elementos que su capacidad. Al analizarlo con *JaTyC*, notamos que efectivamente no reporta ningún error, pero al ejecutarlo falla con una `ArrayIndexOutOfBoundsException`. Esto responde a lo que queríamos indagar: incluso con un protocolo bien definido y un cliente que lo respeta, *JaTyC* no asegura que el objeto esté implementado correctamente. Un programador podría confiarse en su código, al utilizar `isFull()` antes de apilar y no recibir errores de la herramienta, pero, como muestra el ejemplo, nada impide que `isFull()` devuelva cualquier valor.

Por otro lado, si se modifica el método `isFull()` para que controle correctamente cuándo se puede apilar, como en la Figura 4.8, el `Stack` y su protocolo constituyen un ejemplo de cómo podemos apoyarnos en la herramienta para que el código sea más seguro. Al obligar a los clientes a utilizar métodos de control antes de cada operación, conseguimos determinar estáticamente con *JaTyC* que estas ocurran en el momento correcto, reduciendo potencialmente la cantidad de comportamientos inesperados.

Estos experimentos muestran una de las limitaciones aparentes de *JaTyC* y del análisis de *object protocols* con *typestates*. Tomemos como ejemplo nuestro `Stack`, donde en esencia buscamos asegurarnos de que no se apile si está lleno ni se desapile si está vacío. Si observamos el protocolo de la pila, notamos que todas las operaciones están resguardadas detrás de métodos de control. Esto es intencional: por la naturaleza de la herramienta, no hay mecanismos para contar la cantidad de llamadas a métodos ni sus secuencias. En otras palabras, dado que el protocolo se interpreta y procesa como un autómata finito sin pila, *JaTyC* no tiene forma de conocer el tamaño del `Stack` ni la cantidad de elementos apilados en un momento dado.

Bajo la filosofía y las capacidades de la herramienta, esto se soluciona mediante estos métodos de control, quitando flexibilidad al programador a cambio de seguridad y sim-

```
import jatyc.lib.*;

@Typestate("StackProtocol")
public class Stack {
  private int size;
  private int[] stack;
  private int top;

  public Stack(int size) {
    this.size = size;
    this.stack = new int[size];
    this.top = 0;
  }

  public boolean isFull() {
    return this.top == size;
  }

  public boolean isEmpty() {...}
  public void push(int newValue) {...}
  public int pop() {...}
}
```

Fig. 4.8: Código para un `Stack` con su método de control `isFull()` correctamente implementado.

plicidad en el análisis. Notemos que, de no utilizarlos, *JaTyC* reportará estas carencias incluso si en la práctica nuestro código es seguro. En la Figura 4.9 se aprecia un ejemplo: un código cliente apila un solo elemento en un `Stack` de tamaño 10, una acción completamente segura. Sin embargo, como no respeta el protocolo del `Stack`, el análisis reporta un error. En cierto sentido, desde el punto de vista de la seguridad y la correctitud, *JaTyC* reporta falsos positivos: el código potencialmente está bien, pero la herramienta sugiere al programador ser más defensivo.

```
public class Main {
  public static void main(String args[]) throws Exception {
    Stack stack = new Stack(10);
    stack.push(5);
  }
}
```

```
Main.java:4: error: Cannot call [push] on State{Stack, Init}
        stack.push(5);
                   ^
1 error
```

Fig. 4.9: Código cliente del stack y error reportado por *JaTyC*, dado que no sigue su protocolo.

4.3. Intervención Humana

JaTyC es una herramienta de verificación de *typestates* la cual tiene como objetivo objetivo reducir los errores en programas Java. Para ello, se apoya en los protocolos definidos

en archivos con extensión *.protocol* y verifica estáticamente que las llamadas a métodos de objetos protocolados respeten el orden denotado en ellos. Sin embargo, ¿qué impide que un programador defina incorrectamente un protocolo? ¿Qué consecuencias podría tener esto en el programa final y en los errores reportados, o ignorados, por *JaTyC*? Nos interesa investigar cómo se ve afectada la herramienta al depender de la intervención humana para detectar errores correctamente.

```
typestate Iterator {
  HasNext = {
    boolean hasNext(): <true: Next,
    false: end>
  }
  Next = {
    boolean hasNext(): <true: Next,
    false: end>,
    Object next(): HasNext
  }
}
```

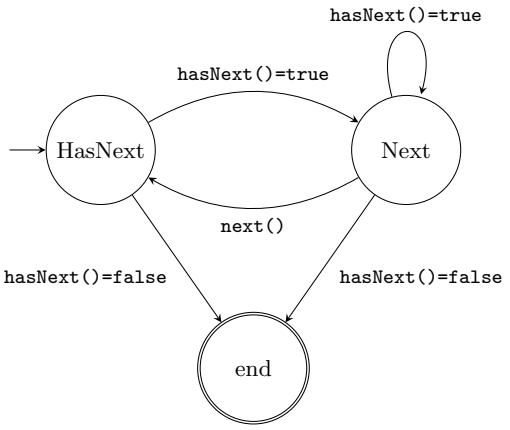


Fig. 4.10: Protocolo `Iterator` y su autómata

```
typestate LoopIterator {
  Init = {
    boolean hasNext(): <true: Init,
    false: Init>,
    Object next(): Init,
    drop: end
  }
}
```

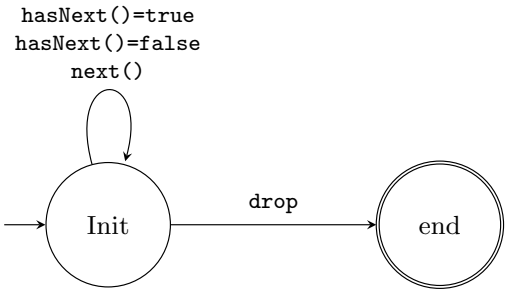


Fig. 4.11: Protocolo `LoopIterator` y su autómata

En primer lugar, realizamos un experimento para probar la "sobreespecificación" de un protocolo, que interpretamos como uno que permite menos cadenas de llamadas a métodos de las esperadas. Lo llevamos a cabo con los dos iteradores introducidos en la sección 3.4 sobre subtipado: la interfaz `Iterator` y la implementación `LoopIterator`. Volvemos a mostrar sus protocolos en las Figuras 4.10 y 4.11. Recordemos que la interfaz permite obtener el siguiente elemento siempre que haya uno, y que su protocolo finaliza cuando no lo hay. Por otro lado, `LoopIterator` recorre la colección de forma cíclica e infinita, por lo que pueden llamarse `next()` y `hasNext()` en cualquier momento. Sus protocolos reflejan este comportamiento.

La prueba realizada utiliza el código cliente de la Figura 4.12, que simplemente instancia un `LoopIterator`, recorre su colección hasta el final y luego solicita un elemento más mediante `next()`. Dado que el protocolo de `LoopIterator` permite llamar a `next()` en cualquier estado, este cliente es válido. Analizamos el código completo en dos versiones: una en la que `LoopIterator` seguía el protocolo de la Figura 4.11, y otra en la que no (y, por lo tanto, seguía el protocolo de su interfaz `Iterator`).

```
import jatyc.lib.*;

public class ClientCode {
  public static void main(String[] args) {
    Object[] arr = {"Hello, World!"};
    LoopIterator it = new LoopIterator(arr);
    while (it.hasNext()) {
      it.next();
    }
    it.next();
  }
}
```

Fig. 4.12: Código cliente para un `LoopIterator` que intenta recorrer cíclicamente la colección.

```
ClientCode2.java:10: error: Cannot call [next] on State{Iterator, end}
                it.next();
                       ^
1 error
```

Fig. 4.13: Error producido por el código cliente de la Figura 4.12 cuando `LoopIterator` no implementa su propio protocolo.

Cuando la clase contaba con su propio protocolo, *JaTyC* no detectó ningún error, como era de esperar. Cuando seguía el protocolo de su interfaz, más restrictivo, la herramienta reportó el error de la Figura 4.13, indicando que la última llamada a `next()` incumplía el protocolo. Este resultado muestra que, si diseñamos los protocolos de forma más restrictiva de lo necesario, se rechazan cadenas de llamadas potencialmente válidas.

El segundo experimento es su contraparte: probamos "subespecificar" el protocolo de un objeto, permitiendo más secuencias de métodos de las esperadas. Para ello construimos la clase `UnderspecifiedIterator` (Figura 4.14), una implementación simple de la interfaz `Iterator` sin comportamientos particulares. Dado que no puede devolver más elementos una vez alcanzado el final de la colección, una nueva llamada a `next()` lanzaría una excepción. Sin embargo, esta clase está sujeta al mismo protocolo que el `LoopIterator` del experimento anterior, que permite llamar a `next()` en cualquier situación.

Como era de esperar, *JaTyC* no generó ningún error al compilar las clases. No obstante, el cliente de la Figura 4.15 intenta obtener elementos dentro de un ciclo sin ninguna guarda, por lo que el iterador termina excediendo el tamaño de la colección y el programa lanza una `java.lang.ArrayIndexOutOfBoundsException`.

Estas pruebas muestran que la herramienta es susceptible al error humano, donde una mala definición del protocolo puede alterar su resultado. Si los protocolos son más restrictivos y "sobreespecifican", se reportan errores para cadenas de métodos que, en principio, querríamos considerar válidas. Si son más laxos y "subespecifican", se permiten cadenas de métodos que querríamos evitar, y las excepciones en tiempo de ejecución pueden interrumpir nuestros programas. Quizás esto no resulte evidente con los ejemplos pequeños que diseñamos, pero en protocolos más complejos, con decenas de *typestates* y aún más transiciones, especificar correctamente el comportamiento esperado de los clientes puede volverse rápidamente difícil y propenso a errores. A futuro, la generación automática de estos protocolos podría ser una solución. Un acercamiento en esta dirección puede verse

```java
import jatyc.lib.*;

@Typestate("UnderspecifiedIterator")
public class UnderspecifiedIterator {
  private int[] items;
  protected int index;

  public UnderspecifiedIterator(int[] items) {
    this.items = items;
    this.index = 0;
  }

  public boolean hasNext() {return this.index < this.items.length;}
  public @Nullable int next() {return this.items[this.index++];}
}
```

```
typestate UnderspecifiedIterator {
  Init = {
    boolean hasNext(): <true: Init, false: Init>,
    int next(): Init,
    drop: end
  }
}
```

Fig. 4.14: Implementación y protocolo de `UnderspecifiedIterator`, una simple implementación de `Iterator`.

```java
import jatyc.lib.*;

public class ClientCode {
  public static void main(String[] args) {
    int[] array = {2,3};
    UnderspecifiedIterator it = new UnderspecifiedIterator(array);
    while (true) {
      it.next();
    }
  }
}
```

Fig. 4.15: Código cliente para un `UnderspecifiedIterator` que intenta obtener elementos dentro de un ciclo sin verificar que existan.

en el trabajo de Uchitel et al. [12], donde se toma la implementación actual de una clase como oráculo.

4.4. Subtipado

4.5. Estado Vs Typestate

4.6. Casos Bordes

4.7. Aliasing

4.8. Excepciones

Bibliografía

[1] Jonathan Aldrich, Joshua Sunshine, Darpan Saini, and Zachary Sparks. Typestate-oriented programming. In *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications*, OOPSLA '09, page 1015–1022, New York, NY, USA, 2009. Association for Computing Machinery.

[2] Amazon Web Services Labs. Crypto Policy Compliance Checker. `https://github.com/awslabs/aws-crypto-policy-compliance-checker`. Accessed: 2026-09-07.

[3] Amazon Web Services Labs. KMS Compliance Checker. `https://github.com/awslabs/aws-kms-compliance-checker`. Accessed: 2026-09-07.

[4] Nels E. Beckman, Duri Kim, and Jonathan Aldrich. An empirical study of object protocols in the wild. In *Proceedings of the 25th European Conference on Object-Oriented Programming*, ECOOP'11, page 2–26, Berlin, Heidelberg, 2011. Springer-Verlag.

[5] Nels E. Beckman, Duri Kim, and Jonathan Aldrich. An empirical study of object protocols in the wild. In *Proceedings of the 25th European Conference on Object-Oriented Programming*, ECOOP'11, page 2–26, Berlin, Heidelberg, 2011. Springer-Verlag.

[6] Kevin Bierhoff and Jonathan Aldrich. Lightweight object specification with typestates. *SIGSOFT Softw. Eng. Notes*, 30(5):217–226, September 2005.

[7] Kevin Bierhoff and Jonathan Aldrich. Modular typestate checking of aliased objects. *SIGPLAN Not.*, 42(10):301–320, October 2007.

[8] Checker Framework. The checker framework. `https://checkerframework.org`. Accessed: 2026-09-07.

[9] Checker Framework. The checker framework official api documentation. `https://checkerframework.org/api/index.html`. Accessed: 2026-11-11.

[10] Checker Framework. The checker framework official manual. `https://checkerframework.org/manual/checker-framework-manual.pdf`. Accessed: 2026-11-12.

[11] Ronald Garcia, Éric Tanter, Roger Wolff, and Jonathan Aldrich. Foundations of typestate-oriented programming. *ACM Trans. Program. Lang. Syst.*, 36(4), October 2014.

[12] Javier Godoy, Juan Pablo Galeotti, Diego Garbervetsky, and Sebastián Uchitel. Enabledness-based testing of object protocols. *ACM Trans. Softw. Eng. Methodol.*, 30(2), January 2021.

[13] JastAdd. JastAdd. `https://jastadd.cs.lth.se/web/`. Accessed: 2026-09-29.

[14] JastAdd. JastAdd - Applications. `https://jastadd.cs.lth.se/web/applications.php`. Accessed: 2026-09-29.

[15] JetBrains. Kotlin language. `https://kotlinlang.org`. Accessed: 2026-09-29.

[16] Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with mungo and stmungo. In *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming*, PPDP '16, page 146–159, New York, NY, USA, 2016. Association for Computing Machinery.

[17] João Mota. Coping with the reality: adding crucial features to a typestate-oriented language. `https://github.com/jdmota/java-typestate-checker/blob/master/docs/msc-thesis.pdf`. Accessed: 2026-10-01.

[18] João Mota and Lorenzo Bacchiani. Jatyc - mungo comparison. `https://github.com/jdmota/java-typestate-checker/wiki/Mungo-comparison`. Accessed: 2026-09-07.

[19] João Mota and Lorenzo Bacchiani. Java typestate checker tool. `https://github.com/jdmota/java-typestate-checker`. Accessed: 2026-09-05.

[20] João Mota and Lorenzo Bacchiani. Java typestate checker tool - documentation. `https://github.com/jdmota/java-typestate-checker/wiki/Documentation`. Accessed: 2026-09-29.

[21] Oracle. Interface Iterator. `https://docs.oracle.com/javase/8/docs/api/java/util/Iterator.html`. Accessed: 2026-09-05.

[22] Oracle. JDK Compiler Module Documentation. `https://docs.oracle.com/en/java/javase/21/docs/api/jdk.compiler/module-summary.html`. Accessed: 2026-11-11.

[23] R E Strom and S Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Trans. Softw. Eng.*, 12(1):157–171, January 1986.

[24] Carnegie Mellon University. Plaid programming language. `https://www.cs.cmu.edu/~aldrich/plaid/`. Accessed: 2026-09-29.

[25] University of Glasgow. Mungo. `https://www.dcs.gla.ac.uk/research/mungo/`. Accessed: 2026-09-07.

[26] University of Glasgow. Mungo - Oficial Repository. `https://bitbucket.org/abcd-glasgow/mungo-tools/src/master/`. Accessed: 2026-10-01.